

PSG Institute of Technology and Applied Research

Name: Madhan Kumaar B S
Email: 23b116@psgitech.ac.in
Roll no: 715523244016
Phone: 8148443863
Branch: PSG iTech
Department: CSBS
Batch: 2027
Degree: B.Tech CSBS

Scan to verify results



2027_IV_CSBS_Operating Systems Lab

PSGITECH_2027_OS_Week 6_COD

Attempt : 1
Total Mark : 20
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Sarah is managing a data center where processes request resources dynamically. Each process has a maximum demand, current allocation, and the system has a limited number of available resources. Sarah wants to detect if the system is in a deadlock state by comparing resource needs against availability.

Write a program to determine whether a deadlock has occurred and identify the processes involved if so.

Input Format

The first line contains an integer n , representing the number of processes.

The second line contains an integer r , representing the number of resource types.

The next n lines each contain r space-separated integers, where the i-th line represents the maximum resources required by process i.

The following n lines each contain r space-separated integers, where the i-th line represents the resources currently allocated to process i.

The last line contains r space-separated integers representing the currently available amount of each resource type in the system.

Output Format

The first line prints "Header "Process Allocation Max Available"

Next n lines: Each line shows the process ID, its allocated resources, its maximum required resources, and the available resources (shown only on the first line).

If a deadlock is detected print "System is in Deadlock and the Deadlock process are"

Followed by the process IDs (e.g., P1, P2, ...) involved in the deadlock, separated by tabs.

If no deadlock is detected print "No Deadlock Occur"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

3

3 6 8

4 3 3

3 4 4

3 3 3

2 0 3

1 2 4

1 2 0

Output: Process Allocation Max Available

P1 3 3 3 3 6 8 1 2 0

P2 2 0 3 4 3 3
P3 1 2 4 3 4 4

System is in Deadlock and the Deadlock process are
P1 P2 P3

Answer

```
#include <stdio.h>
```

```
int main() {  
    int n, r;  
    scanf("%d", &n);  
    scanf("%d", &r);  
  
    int max[n][r];  
    int alloc[n][r];  
    int avail[r];  
    int need[n][r];  
  
    // Read maximum matrix  
    for(int i = 0; i < n; i++) {  
        for(int j = 0; j < r; j++) {  
            scanf("%d", &max[i][j]);  
        }  
    }  
  
    // Read allocation matrix  
    for(int i = 0; i < n; i++) {  
        for(int j = 0; j < r; j++) {  
            scanf("%d", &alloc[i][j]);  
        }  
    }  
  
    // Read available resources  
    for(int j = 0; j < r; j++) {  
        scanf("%d", &avail[j]);  
    }  
  
    // Calculate need matrix  
    for(int i = 0; i < n; i++) {  
        for(int j = 0; j < r; j++) {  
            need[i][j] = max[i][j] - alloc[i][j];  
        }  
    }  
}
```

```

    }
}

// Print header and process information
printf("Process\t Allocation\t Max\t Available\t\n");
for(int i = 0; i < n; i++) {
    printf("P%d\t ", i+1);
    // Print allocation
    for(int j = 0; j < r; j++) {
        printf("%d ", alloc[i][j]);
    }
    printf("\t\t");
    // Print max
    for(int j = 0; j < r; j++) {
        printf("%d ", max[i][j]);
    }
    // Print available (only for first process)
    if(i == 0) {
        printf("\t ");
        for(int j = 0; j < r; j++) {
            printf("%d ", avail[j]);
        }
    }
    printf("\n");
}
printf("\n");

// Deadlock detection
int finish[n];
for(int i = 0; i < n; i++) {
    finish[i] = 0;
}

int work[r];
for(int j = 0; j < r; j++) {
    work[j] = avail[j];
}

int count = 0;
while(count < n) {
    int found = 0;
    for(int i = 0; i < n; i++) {

```

```

        if(!finish[i]) {
            int j;
            for(j = 0; j < r; j++) {
                if(need[i][j] > work[j]) {
                    break;
                }
            }
            if(j == r) {
                for(int k = 0; k < r; k++) {
                    work[k] += alloc[i][k];
                }
                finish[i] = 1;
                found = 1;
                count++;
            }
        }
    }
    if(!found) {
        break;
    }
}

```

```

// Check for deadlock
int deadlock = 0;
for(int i = 0; i < n; i++) {
    if(!finish[i]) {
        deadlock = 1;
        break;
    }
}

```

```

if(deadlock) {
    printf("System is in Deadlock and the Deadlock process are\n");
    for(int i = 0; i < n; i++) {
        if(!finish[i]) {
            printf("P%d\t", i+1);
        }
    }
    printf("\n");
} else {
    printf("No Deadlock Occur\n");
}

```

```
} return 0;
```

Status : Correct

Marks : 10/10

2. Problem Statement

Design a program to determine if a given system is in a safe state or not by implementing the Banker's algorithm. The system has N processes and M resources. For each process, the program takes the allocation matrix and maximum matrix from the user. It then takes the available resources vector from the user.

The program then calculates the need matrix for each process and checks if the system is in a safe state by finding a safe sequence of processes. If a safe sequence exists, it is printed out. Otherwise, the program outputs that the system is not safe.

Input Format

The first line contains an integer, N, representing the number of processes.

The second line contains an integer, M, representing the number of resources.

The next N lines contain M integers each, representing the allocation matrix.

The next N lines contain M integers each, representing the maximum demand matrix.

The last line contains M integers, representing the available resources vector.

Output Format

If a safe sequence of processes exists, print "Following is the SAFE Sequence" followed by the sequence of processes in the order they should be executed, separated by "->".

If a safe sequence does not exist, print "The following system is not safe".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

3

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

3 3 2

Output: Following is the SAFE Sequence

P1 -> P3 -> P4 -> P0 -> P2

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX 5
```

```
bool isSafe(int N, int M, int allocation[][MAX], int max[][MAX], int available[], int  
safeSeq[]) {
```

```
    int need[N][M];
```

```
    bool finish[N];
```

```
    int work[M];
```

```
    int count = 0;
```

```
    for (int i = 0; i < N; i++) {
```

```
        for (int j = 0; j < M; j++) {
```

```
            need[i][j] = max[i][j] - allocation[i][j];
```

```
        }
```

```
        finish[i] = false;
```

```
    }
```

```
    for (int i = 0; i < M; i++) {
```

```

        work[i] = available[i];
    }

    while (count < N) {
        bool found = false;
        for (int p = 0; p < N; p++) {
            if (!finish[p]) {
                int j;
                for (j = 0; j < M; j++) {
                    if (need[p][j] > work[j]) {
                        break;
                    }
                }

                if (j == M) {
                    for (int k = 0; k < M; k++) {
                        work[k] += allocation[p][k];
                    }

                    safeSeq[count++] = p;
                    finish[p] = true;
                    found = true;
                }
            }
        }

        if (!found) {
            return false;
        }
    }

    return true;
}

int main() {
    int N, M;

    scanf("%d", &N);
    scanf("%d", &M);

    int allocation[N][MAX], max[N][MAX], available[MAX], safeSeq[N];

```



```

for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        scanf("%d", &allocation[i][j]);
    }
}

for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        scanf("%d", &max[i][j]);
    }
}

for (int i = 0; i < M; i++) {
    scanf("%d", &available[i]);
}

if (isSafe(N, M, allocation, max, available, safeSeq)) {
    printf("Following is the SAFE Sequence\n");
    for (int i = 0; i < N; i++) {
        printf(" P%d", safeSeq[i]);
        if (i != N - 1) {
            printf(" ->");
        }
    }
    printf("\n");
} else {
    printf("The following system is not safe\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10